

Program 1:

1. Pseudocode

START

FUNCTION hitung_kata(teks):

kata $\leftarrow$ pisahkan teks berdasarkan spasi

RETURN panjang(kata)

FUNCTION hitung_kalimat(teks):

bagian $\leftarrow$ pisahkan teks berdasarkan "."

jumlah $\leftarrow$ hitung bagian yang tidak kosong (bagian $\neq$ "")

RETURN jumlah

INPUT teks

jumlah_kata $\leftarrow$ hitung_kata(teks)

jumlah_kalimat $\leftarrow$ hitung_kalimat(teks)

OUTPUT "Teks tersebut memuat " + jumlah_kalimat + " kalimat dan " +
jumlah_kata + " kata."

END

2. Flowchart



3. Python

#Menghitung jumlah kata dan jumlah kalimat

#Memisahkan teks berdasarkan spasi

def hitung_kata(teks):

 kata = teks.split()

 return len(kata)

#Memisahkan teks berdasarkan tanda titik

def hitung_kalimat(teks):

 bagian = teks.split(".")

 jumlah = len([b for b in bagian if b.strip() != ""]) #Menyaring bagian kosong

 return jumlah

#Input dari pengguna

teks = input("Masukkan teks: ")

#Proses perhitungan

jumlah_kata = hitung_kata(teks)

jumlah_kalimat = hitung_kalimat(teks)

#Output hasil

print(f'Teks tersebut memuat {jumlah_kalimat} kalimat dan {jumlah_kata} kata.")

4. R

#Program 1: Analisis Teks - Menghitung Jumlah Kata dan Jumlah Kalimat

```
hitung_kata <- function(teks) {  
  #Memisahkan teks berdasarkan spasi lalu menghitung elemen  
  kata <- unlist(strsplit(trimws(teks), "\\s+"))  
  return(length(kata))  
}  
  
hitung_kalimat <- function(teks) {  
  
  #Memisahkan teks berdasarkan tanda titik, lalu hitung bagian tidak kosong  
  bagian <- strsplit(teks, "\\." )[[1]]  
  jumlah <- length(bagian[trimws(bagian) != ""])  
  return(jumlah)  
}  
  
#Input dari pengguna  
teks <- readline(prompt = "Masukkan teks: ")  
  
#Proses perhitungan  
jumlah_kata <- hitung_kata(teks)  
jumlah_kalimat <- hitung_kalimat(teks)  
  
#Output hasil  
print(paste0("Teks tersebut memuat ", jumlah_kalimat, " kalimat dan ",  
jumlah_kata, " kata."))
```

Program 2:

1. Pseudocode

START

Buat K1 = "Saya tak 'kan menyerah."

Buat K2 = "Ia berkata, ""Aku menyayangimu.""

Buat K3 = ""Coba jelaskan pengertian 'cross-validation' dalam Machine Learning!""

Buat K4 = "Surat keputusan itu bernomor 62/UN.34/19/2023."

Tampilkan K1

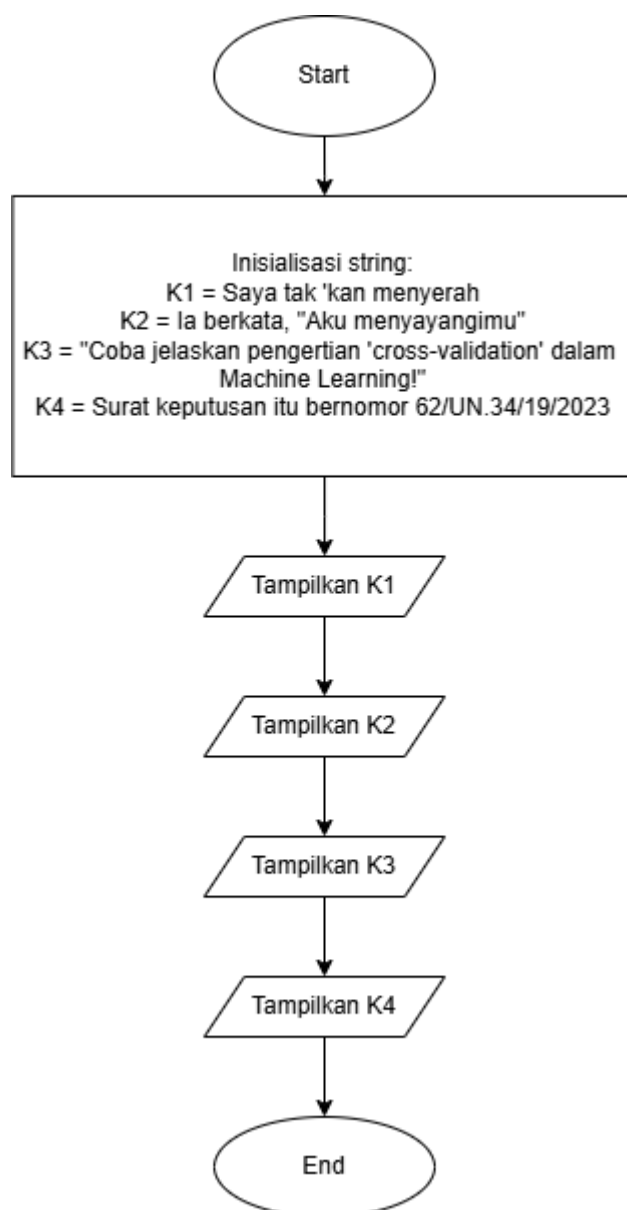
Tampilkan K2

Tampilkan K3

Tampilkan K4

END

2. Flowchart



3. Python

```
K1 = "Saya tak 'kan menyerah."  
K2 = 'Ia berkata, "Aku menyayangimu."'"  
K3 = "\Coba jelaskan pengertian 'cross-validation' dalam Machine  
Learning!\\"""  
K4 = "Surat keputusan itu bernomor 62/UN.34/19/2023."  
  
print (K1)  
print (K2)  
print (K3)  
print (K4)
```

4. R

```
K1 <- "Saya tak 'kan menyerah."  
K2 <- 'Ia berkata, "Aku menyayangimu."'"  
K3 <- "\"Coba jelaskan pengertian 'cross-validation' dalam Machine  
Learning!\\"""  
K4 <- "Surat keputusan itu bernomor 62/UN.34/19/2023."  
  
cat(K1, "\n")  
cat(K2, "\n")  
cat(K3, "\n")  
cat(K4, "\n")
```

Program 3:

1. Pseudocode

START

FUNCTION hitung_akar(a, b, c):

$d \leftarrow b^2 - 4 \times a \times c$

IF $d > 0$ THEN

$x1 \leftarrow (-b + \sqrt{d}) / (2a)$

$x2 \leftarrow (-b - \sqrt{d}) / (2a)$

OUTPUT "Persamaan Memiliki Dua Akar Real yang Berbeda"

OUTPUT x1 (3 desimal)

OUTPUT x2 (3 desimal)

ELSE IF $d = 0$ THEN

$x \leftarrow -b / (2a)$

OUTPUT "Persamaan Memiliki Akar Real Kembar"

OUTPUT $x_1 = x_2$ (3 desimal)

ELSE

OUTPUT "Persamaan Memiliki Akar-Akar Imajiner"

END IF

END FUNCTION

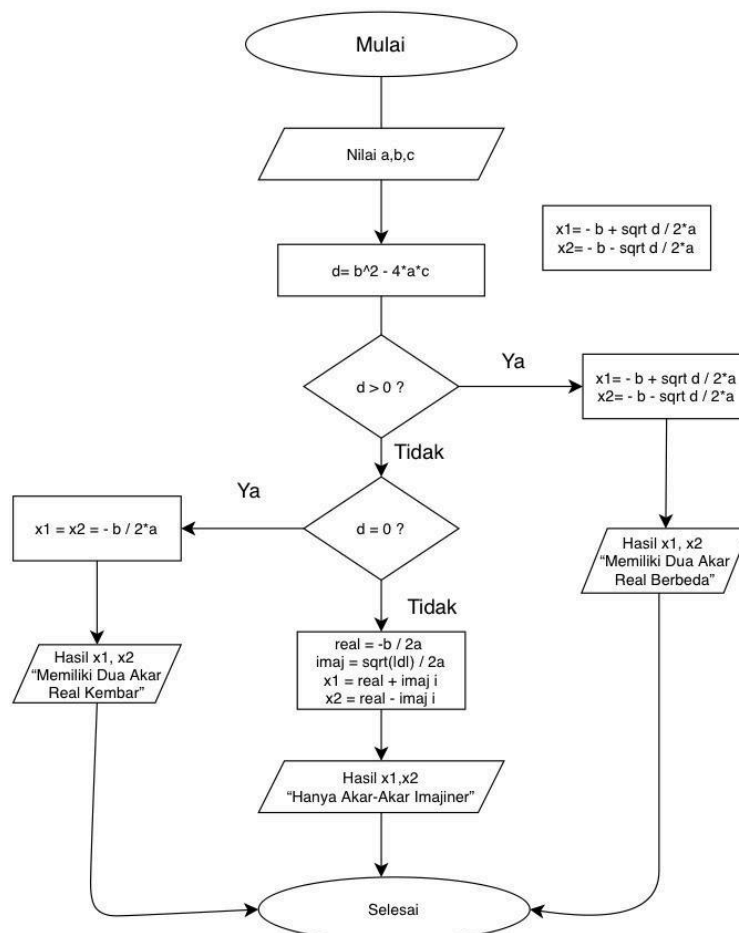
INPUT a, b, c

OUTPUT "Persamaan: $a x^2 + b x + c = 0$ "

CALL hitung_akar(a, b, c)

END

2. Flowchart



$$x_1 = -1.000 + 2.000 i$$

$$x_2 = -1.000 - 2.000 i$$

-
- Kondisi 1
 - a: 1
 - b: -5
 - c: 6
 - Persamaan: $1 x^2 + -5 x + 6 = 0$
 - Persamaan Memiliki Dua Akar Real yang Berbeda:
 - $x_1 = 3.000$
 - $x_2 = 2.000$
- Kondisi 2
 - a: 1
 - b: -6
 - c: 9
 - Persamaan: $1 x^2 + -6 x + 9 = 0$
 - Persamaan Memiliki Akar Real Kembar:
 - $x_1 = x_2 = 3.000$
- Kondisi 3
 - a: 1
 - b: 2
 - c: 5
 - Persamaan: $1 x^2 + 2 x + 5 = 0$
 - Persamaan Memiliki Akar-Akar Imajiner:
 - $x_1 = -1.000 + 2.000 i$
 - $x_2 = -1.000 - 2.000 i$

3. Python

#Program 3

#Fungsi Akar-Akar Persamaan Kuadrat

def hitung_akar(a, b, c):

 d = b**2 - 4*a*c

 if d > 0:

 print("Persamaan Memiliki Dua Akar Real yang Berbeda:")

 print("x1 =", round((-b + math.sqrt(d)) / (2*a),3))

 print("x2 =", round((-b - math.sqrt(d)) / (2*a),3))

 elif d == 0:

 x = -b / (2*a)

 print("Persamaan Memiliki Akar Real Kembar:")

```

    print("x1 = x2 =", round(x),3)
else:
    real = -b / (2*a)
    imaj = math.sqrt(abs(d)) / (2*a)
    print("Persamaan Akar-Akar Imajiner:")
    print("x1 =", round(real, 3), "+", round(imaj, 3), "i")
    print("x2 =", round(real, 3), "-", round(imaj, 3), "i")

#Memasukkan Nilai Variabel dari Persamaan Kuadrat
a = float(input("Masukkan nilai a: "))
b = float(input("Masukkan nilai b: "))
c = float(input("Masukkan nilai c: "))

#Menampilkan Bentuk Persamaan
print("Persamaan:", a, "x² +", b, "x +", c, "= 0")

#Menampilkan Hasil
hitung_akar(a, b, c)

```

4. R

```

#Program 3
#Fungsi Akar-akar Persamaan Kuadrat
hitung_akar <- function(a, b, c) {
  diskriminan <- b^2 - 4*a*c
  if (diskriminan > 0) {
    x1 <- (-b + sqrt(diskriminan)) / (2*a)
    x2 <- (-b - sqrt(diskriminan)) / (2*a)
    cat("Persamaan Memiliki Dua Akar Real yang Berbeda:\n")
    cat("x1 =", sprintf("%.3f", x1), "\n")
    cat("x2 =", sprintf("%.3f", x2), "\n")
  } else if (diskriminan == 0) {
    x <- -b / (2*a)
    cat("Persamaan Memiliki Akar Real Kembar:\n")
    cat("x1 = x2 =", sprintf("%.3f", x), "\n")
  } else {

```



```

    real <- -b / (2*a)
    imaj <- sqrt(abs(diskriminan)) / (2*a)
    cat("Persamaan Memiliki Akar-Akar Imajiner:\n")
    cat("x1 =", sprintf("%.3f", real), "+", sprintf("%.3f", imaj), "i\n")
    cat("x2 =", sprintf("%.3f", real), "-", sprintf("%.3f", imaj), "i\n")
  }
}

#Memasukkan Nilai Variabel dari Persamaan Kuadrat
a <- as.numeric(readline("Masukkan nilai a: "))
b <- as.numeric(readline("Masukkan nilai b: "))
c <- as.numeric(readline("Masukkan nilai c: "))

#Menampilkan Bentuk Persamaan
cat("Persamaan:", a, "x2 +", b, "x +", c, "= 0\n")

#Menampilkan Hasil
hitung_akar(a, b, c)

```

Program 4

1. Pseudocode

START

INPUT nip

tahun ← potong nip dari indeks 0 hingga sebelum 4

bulan ← potong nip dari indeks 5 hingga sebelum 6

tanggal ← potong nip dari indeks 7 hingga sebelum 8

tahun_angka ← konversi ke angka (tahun)

tanggal_angka ← konversi ke angka (tanggal)

IF bulan == "01" THEN

 bulan_nama ← "Januari"; hari_maks ← 31

ELSE IF bulan == "02" THEN

```

    bulan_nama ← "Februari"
    IF (tahun_angka MOD 4 == 0 AND tahun_angka MOD 100 != 0) OR
(tahun_angka MOD 400 == 0) THEN
        hari_maks ← 29
    ELSE
        hari_maks ← 28
    ENDIF
ELSE IF bulan == "03" THEN
    bulan_nama ← "Maret"; hari_maks ← 31
ELSE IF bulan == "04" THEN
    bulan_nama ← "April"; hari_maks ← 30
ELSE IF bulan == "05" THEN
    bulan_nama ← "Mei"; hari_maks ← 31
ELSE IF bulan == "06" THEN
    bulan_nama ← "Juni"; hari_maks ← 30
ELSE IF bulan == "07" THEN
    bulan_nama ← "Juli"; hari_maks ← 31
ELSE IF bulan == "08" THEN
    bulan_nama ← "Agustus"; hari_maks ← 31
ELSE IF bulan == "09" THEN
    bulan_nama ← "September"; hari_maks ← 30
ELSE IF bulan == "10" THEN
    bulan_nama ← "Oktober"; hari_maks ← 31
ELSE IF bulan == "11" THEN
    bulan_nama ← "November"; hari_maks ← 30
ELSE IF bulan == "12" THEN
    bulan_nama ← "Desember"; hari_maks ← 31
ELSE
    bulan_nama ← "Bulan tidak valid"
END IF

IF tanggal_angka < 1 OR tanggal_angka > hari_maks THEN
    tanggal_status ← "Tanggal tidak valid"
ELSE

```

```

tanggal_status ← tanggal
ENDIF

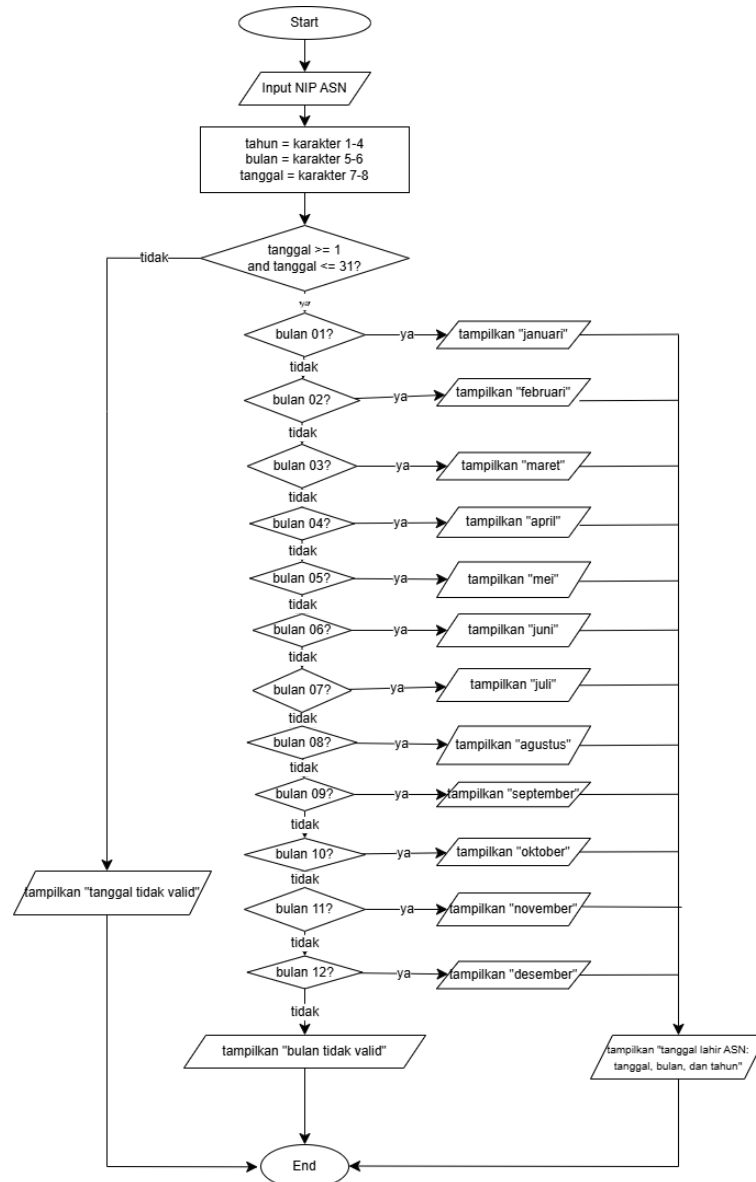
```

```

OUTPUT "Tanggal lahir ASN:", tanggal_status, bulan_nama, tahun
END

```

2. Flowchart



3. Python

```

# Menerima input NIP dari pengguna
nip = input("Masukkan NIP ASN:")

# Ekstrak tahun, bulan, dan tanggal menggunakan slicing (semuanya bertipe
string)

```

```
tahun = nip[0:4]
bulan = nip[4:6]
tanggal = nip[6:8]
```

```
#Percabangan untuk membatasi tanggal
if int(tanggal) >= 1 and int(tanggal) <= 31:
    tanggal_lahir = tanggal
else:
    tanggal_lahir = "Tanggal lahir tidak valid"
```

```
#Percabangan untuk menentukan nama bulan menggunakan perbandingan string
if bulan == "01":
    nama_bulan = "Januari"
elif bulan == "02":
    nama_bulan = "Februari"
elif bulan == "03":
    nama_bulan = "Maret"
elif bulan == "04":
    nama_bulan = "April"
elif bulan == "05":
    nama_bulan = "Mei"
elif bulan == "06":
    nama_bulan = "Juni"
elif bulan == "07":
    nama_bulan = "Juli"
elif bulan == "08":
    nama_bulan = "Agustus"
elif bulan == "09":
    nama_bulan = "September"
elif bulan == "10":
    nama_bulan = "Oktober"
elif bulan == "11":
    nama_bulan = "November"
elif bulan == "12":
```

```

    nama_bulan = "Desember"
else:
    nama_bulan = "Bulan tidak valid"

# Tampilkan hasil akhir ke layar
print(f"Tanggal lahir ASN: {tanggal_lahir} {nama_bulan} {tahun}")

```

4. R

```

#Menerima input NIP
nip <- readline(prompt = "Masukkan NIP ASN:")

#Ekstrak tahun, bulan, dan tanggal menggunakan substr (semuanya bertipe
string)
tahun <- substr(nip, 1, 4)
kode <- substr(nip, 5, 6)
tanggal <- substr(nip, 7, 8)

#Struktur if-else untuk membatasi tanggal
tgl_angka <- as.integer(tanggal)

if(!is.na(tgl_angka) & tgl_angka >= 1 & tgl_angka <= 31) {
  tanggal_ok <- "valid"
} else {
  tanggal_ok <- "error"
}

#Struktur percabangan if-else bertingkat untuk menentukan nama bulan
if (kode == "01") {
  bulan_nama <- "Januari"
} else if (kode == "02") {
  bulan_nama <- "Februari"
} else if (kode == "03") {
  bulan_nama <- "Maret"
} else if (kode == "04") {

```

```

    bulan_nama <- "April"
} else if (kode == "05") {
    bulan_nama <- "Mei"
} else if (kode == "06") {
    bulan_nama <- "Juni"
} else if (kode == "07") {
    bulan_nama <- "Juli"
} else if (kode == "08") {
    bulan_nama <- "Agustus"
} else if (kode == "09") {
    bulan_nama <- "September"
} else if (kode == "10") {
    bulan_nama <- "Oktober"
} else if (kode == "11") {
    bulan_nama <- "November"
} else if (kode == "12") {
    bulan_nama <- "Desember"
} else {
    bulan_nama <- "Bulan tidak valid"
}

# Menampilkan hasil akhir
cat("Tanggal lahir ASN:", tanggal, bulan_nama, tahun, "\n")

```

Program 5

1. Pseudocode

ALGORITMA Penentuan_Cluster

DEKLARASI

$A \leftarrow (2, 1, 3)$

$B \leftarrow (1, -4, 6)$

$C \leftarrow (-2, 3, -2)$

$x_1, x_2, x_3 : \text{real}$

dA, dB, dC : real

DESKRIPSI

INPUT x1, x2, x3

IF x1, x2, atau x3 bukan numerik THEN

 OUTPUT "input tidak valid"

ELSE

$U \leftarrow (x1, x2, x3)$

FUNCTION jarak(P, Q)

 RETURN $\sqrt{(P1-Q1)^2 + (P2-Q2)^2 + (P3-Q3)^2}$

END FUNCTION

dA $\leftarrow$ jarak(U, A)

dB $\leftarrow$ jarak(U, B)

dC $\leftarrow$ jarak(U, C)

OUTPUT dA, dB, dC

IF dA = dB OR dA = dC OR dB = dC THEN

 IF dA = dB AND dA = dC THEN

 OUTPUT "Jarak sama: A = B = C"

 ELSE IF dA = dB THEN

 OUTPUT "Jarak sama: A = B"

 ELSE IF dA = dC THEN

 OUTPUT "Jarak sama: A = C"

 ELSE IF dB = dC THEN

 OUTPUT "Jarak sama: B = C"

 ENDIF

ELSE IF dA < dB AND dA < dC THEN

 OUTPUT "Titik U masuk cluster A"

ELSE IF dB < dA AND dB < dC THEN

OUTPUT "Titik U masuk cluster B"

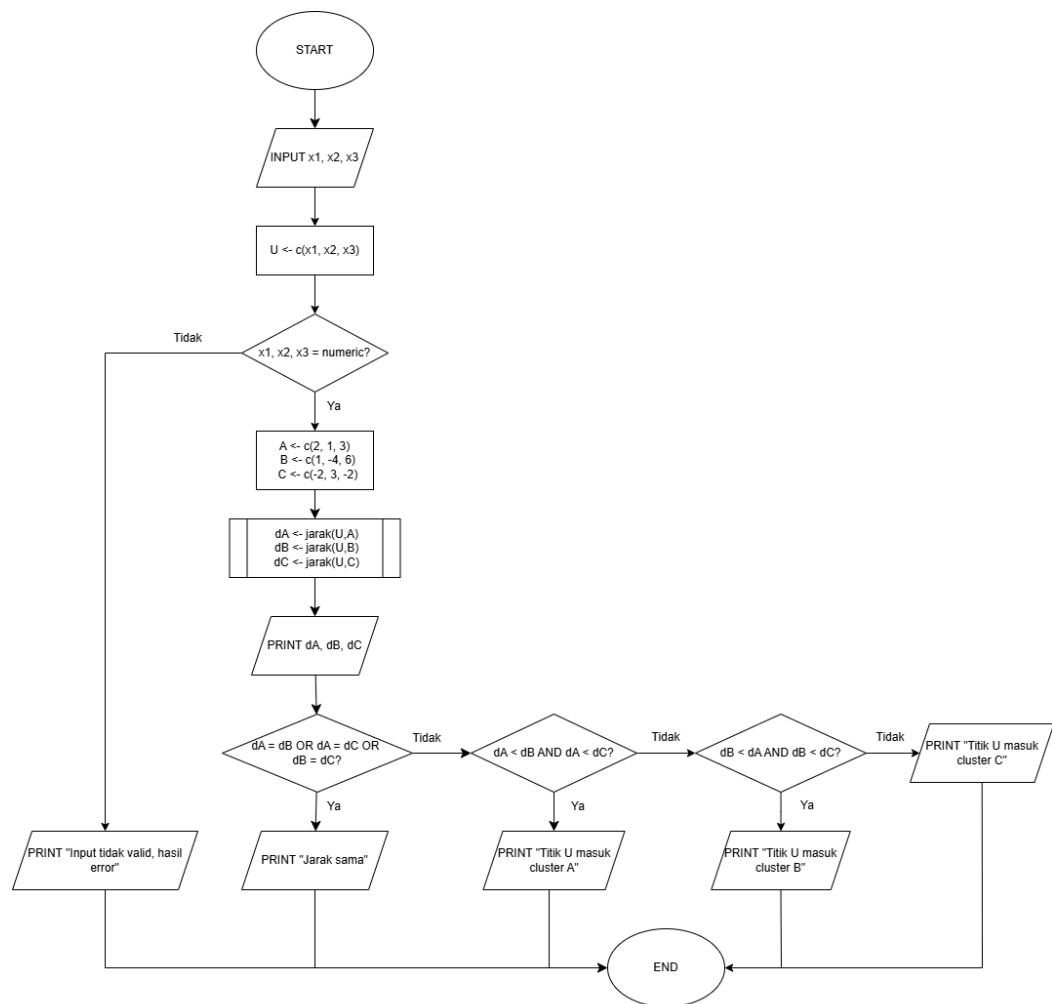
ELSE IF $dC < dA$ AND $dC < dB$ THEN

OUTPUT "Titik U masuk cluster C"

ENDIF

END ALGORITMA

2. Flowchart



Sub-Program

Pendefinisian dari fungsi jarak

	jarak(P,Q)	
	RETURN sqrt((P1-Q1) ² +(P2-Q2) ² +(P3-Q3) ²)	

3. Python

```
import math
```



```

# input + validasi
try:
    x1 = float(input("Masukkan x1: "))
    x2 = float(input("Masukkan x2: "))
    x3 = float(input("Masukkan x3: "))
except:
    print("input tidak valid")
    exit()

U = (x1, x2, x3)

# titik pusat cluster
A = (2, 1, 3)
B = (1, -4, 6)
C = (-2, 3, -2)

# fungsi jarak
def jarak(P, Q):
    return math.sqrt((P[0]-Q[0])**2 + (P[1]-Q[1])**2 + (P[2]-Q[2])**2)

# hitung jarak
dA = jarak(U, A)
dB = jarak(U, B)
dC = jarak(U, C)

# tampilkan jarak
print("Jarak ke A:", dA)
print("Jarak ke B:", dB)
print("Jarak ke C:", dC)

# kondisi
if dA == dB or dA == dC or dB == dC:
    if dA == dB and dA == dC:
        print("Jarak sama: A = B = C")
    elif dA == dB:
        print("Jarak sama: A = B")

```

```

elif dA == dC:
    print("Jarak sama: A = C")
elif dB == dC:
    print("Jarak sama: B = C")

elif dA < dB and dA < dC:
    print("Titik U masuk cluster A")

elif dB < dA and dB < dC:
    print("Titik U masuk cluster B")

elif dC < dA and dC < dB:
    print("Titik U masuk cluster C")

```

4. R

```

# input titik U
x1 <- as.numeric(readline("Masukkan x1: "))
x2 <- as.numeric(readline("Masukkan x2: "))
x3 <- as.numeric(readline("Masukkan x3: "))

U <- c(x1, x2, x3)

if (is.na(x1) || is.na(x2) || is.na(x3)) {
    cat("Input tidak valid, hasil error\n")
} else {

# titik pusat cluster
A <- c(2, 1, 3)
B <- c(1, -4, 6)
C <- c(-2, 3, -2)

# fungsi jarak
jarak <- function(P, Q) {
    sqrt((P[1]-Q[1])^2 + (P[2]-Q[2])^2 + (P[3]-Q[3])^2)
}

```

```

}

# hitung jarak
dA <- jarak(U, A)
dB <- jarak(U, B)
dC <- jarak(U, C)

# tampilkan jarak
cat("Jarak ke A:", dA, "\n")
cat("Jarak ke B:", dB, "\n")
cat("Jarak ke C:", dC, "\n")

# menentukan cluster
if (dA == dB || dA == dC || dB == dC) {

  if (dA == dB && dA == dC) {
    cat("Jarak sama: A = B = C\n")
  } else if (dA == dB) {
    cat("Jarak sama: Titik U masuk cluster A dan B\n")
  } else if (dA == dC) {
    cat("Jarak sama: Titik U masuk cluster A dan C\n")
  } else if (dB == dC) {
    cat("Jarak sama: Titik U masuk cluster B dan C\n")
  }

} else if (dA < dB && dA < dC) {
  cat("Titik U masuk cluster A\n")
} else if (dB < dA && dB < dC) {
  cat("Titik U masuk cluster B\n")
} else {
  cat("Titik U masuk cluster C\n")
}

}

```

Program 6

1. Pseudocode

START

FUNCTION interval_konfidensi(p_topi, n, alpha):

IF p_topi < 0 OR p_topi > 1 THEN

 OUTPUT "Error: proporsi harus berada pada rentang 0 dan 1"

 RETURN

END IF

IF alpha = 0.10 THEN

$z \leftarrow 1.645$

ELSE IF alpha = 0.05 THEN

$z \leftarrow 1.96$

ELSE

 OUTPUT "Error: nilai alpha harus 0.05 atau 0.10"

 RETURN

END IF

$\text{margin_error} \leftarrow z \times \sqrt{(p_topi \times (1 - p_topi) / n)}$

$\text{batas_bawah} \leftarrow p_topi - \text{margin_error}$

$\text{batas_atas} \leftarrow p_topi + \text{margin_error}$

OUTPUT "batas bawah = " batas_bawah

OUTPUT "batas atas = " batas_atas

OUTPUT "interval konfidensi = (" batas_bawah ", " batas_atas ")"

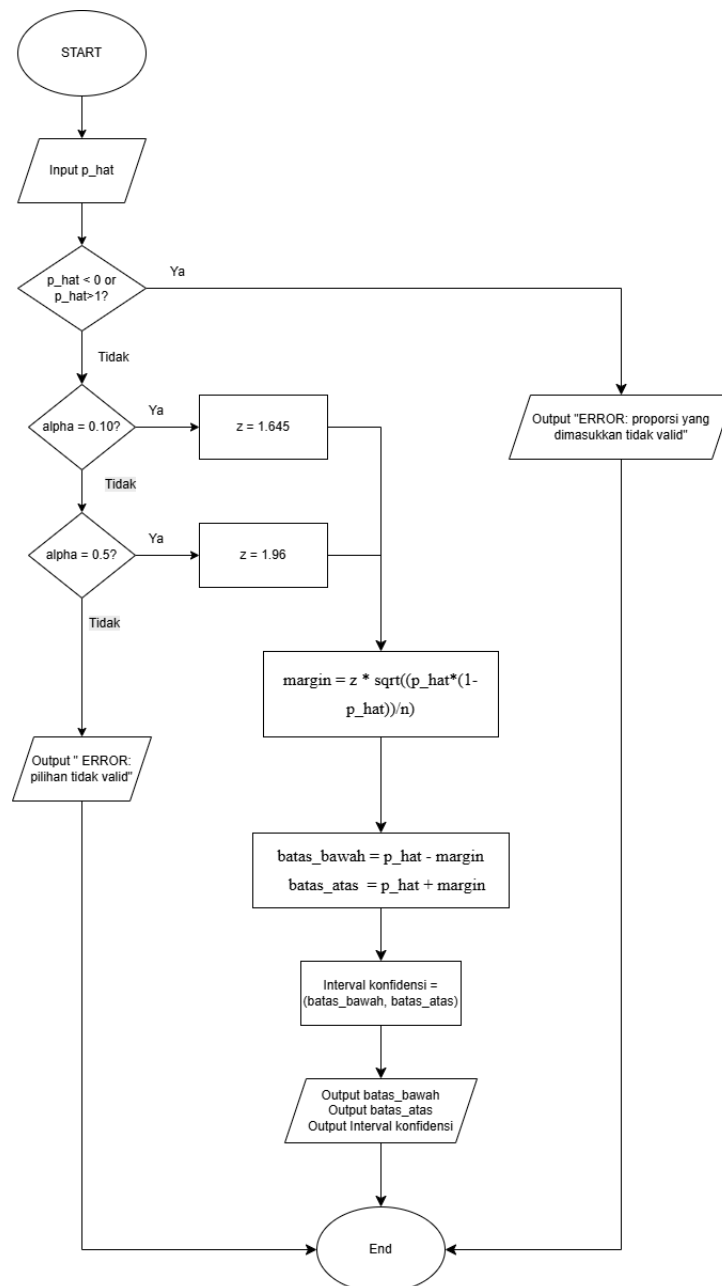
END FUNCTION

INPUT p_topi, n, alpha

CALL interval_konfidensi(p_topi, n, alpha)

END

2. Flowchart



3. Python

4. R